Naïve Bayes Classification

DSC 340 Applied Machine Learning

Spring 2025

The Plan

- How do we model language?
- Introduce naïve Bayes classification
- Discuss probabilities along the way

NLP

Natural Language Processing

NLP

- **Natural Language Processing** (**NLP**)
- The use of computing and artificial intelligence to analyze, understand, and generate human language.
- Applications that involve NLP:
  - Google Translate
  - Spam detectors
  - Customer support chatbots
  - Sentiment analysis of social media posts

Natural Language Processing combines **linguistics**, **statistics**, and **machine learning** to tackle tasks like translating languages, classifying text, or generating human-like responses. It's a field that bridges **computers** and **human communication**.

NLP powers everything from search engines to chatbots to translation tools. It's how machines try to process, analyze, and generate language, even with its inherent complexity.

We'll start by looking at what makes NLP so challenging and how early methods addressed those challenges.

Why is NLP difficult?

- **Ambiguity**: Words and phrases often have multiple meanings
- **Context Dependence**: Understanding of language depends on shared knowledge and cultural references
- **Evolving Language**: Idioms, slang, and meaning change over time
- **Data Sparsity**: Rare expressions are a challenge to modelling
- **Lack of Grounding**: Machines don't have real-world experiences

Natural **language** is incredibly complex, and these challenges highlight why **NLP** is so difficult. Unlike formal languages, natural languages are full of **ambiguities**, depend heavily on **context**, and evolve constantly. Machines don’t have the shared knowledge or experiences we use to interpret meaning, which is why tasks like understanding idioms or nuanced expressions are so hard.

Bag of Words (BoW)

- Counts word occurrences
- Ignores order
- Easy to implement, forms basis for many methods



The **Bag of Words (BoW)** model is one of the simplest methods used in **NLP**. It represents text as a collection—or "bag"—of individual words, ignoring grammar and word order. The idea is to count how often each word appears in a document.

For example, if we have two sentences: “I love Shakespeare” and “He loves Shakespeare,” the BoW representation would count the occurrences of each word and create vectors.

The structure is simple but useful for tasks like document comparison or **text classification**.

One limitation of this approach is that it doesn’t capture the context or sequence of words. So, while it’s easy to implement and often a starting point, it’s not ideal for more complex tasks that require an understanding of relationships between words.

Let’s move to an example of how this works.

Simple Example: BoW

- Document 1:
  "I love Shakespeare" → `[I:1, love:1, Shakespeare:1]`
- Document 2:
  "He loves Shakespeare" → `[He:1, loves:1, Shakespeare:1]`
- Compare vectors → measure similarity
- Trained on **corpus**

Here's an example of how the **Bag of Words** model works in practice. We have two sentences: "I love Shakespeare" and "He loves Shakespeare."
For each document, we count how often each word appears. The result is a vector for each document.

By comparing these vectors, we can measure **similarity** between documents. For example, both documents mention "Shakespeare," so they're likely related.

Even without coding, this example shows how text can be reduced to a simple, structured format that machines can process. While this approach is foundational, it's limited because it doesn't consider word order or context—"love" and "loves" are treated as separate words.

N-Grams and Markov Chains

- Predict next word from last n words
  - Unigrams (n=1)
  - Bigrams (n=2)
  - Trigrams (n=3)
- Chain of probabilities
- Good for rudimentary text generation

N-grams extend the **Bag of Words** concept by looking at sequences of words instead of treating each word as independent. An **n-gram** is simply a group of **n** consecutive words in a text. For example:

- With n=2 (bigrams), the sentence “I love Shakespeare” becomes: ["I love," "love Shakespeare"].
- With n=3 (trigrams), it becomes: ["I love Shakespeare"].

This approach captures some context by preserving the order of words within the **n-grams**, unlike Bag of Words. It’s commonly used for tasks like **text prediction**, where knowing the previous word(s) helps predict the next.

N-grams form the basis for many early NLP systems, including **Markov Chains**, which use probabilities to predict the next word in a sequence based on the last **n** words. However, like Bag of Words, they have limitations. They don’t capture long-range dependencies or deeper meaning.

Example: Markov Chain Text

- Predicts the next word based on the last n words
- Uses probabilities derived from text patterns
- Mimics style but lacks meaning or understanding

Markov Chains use **n-grams** to predict the next word in a sequence based on probabilities. These probabilities are calculated by analyzing patterns in a large dataset. For instance, in Shakespearean text, the word “to” might frequently be followed by “be,” giving it a high probability.

When used to generate text, Markov Chains can mimic the style of the original material, but the results are often nonsensical. For example, after analyzing Shakespeare, it might produce something like: “To be or not to be, that is the banana.”

This randomness highlights an important limitation: Markov Chains don’t understand meaning. They’re working entirely on probabilities without any grasp of **context** or **semantics**. Despite their simplicity, Markov Chains were foundational in early NLP, helping researchers explore how machines could generate text.

Clustering Documents

- Grouping texts by word frequency (BoW)
- Example: identifying topics, genres, or authors

Clustering groups documents based on shared features, like the frequency of words. For example, documents with words like “goal,” “team,” and “score” might cluster together as sports-related. This method doesn’t require labels, making it useful for finding patterns in unstructured data.

Classification

- Similar to clustering but with predefined labels
- Examples: Spam detection, language identification

Classification assigns a category to each document based on patterns it recognizes. For instance, a classifier could determine whether an email is spam or identify the language of a text. Unlike clustering, it uses labeled data to learn from examples.

Applications in Action

- Search engines (pre-PageRank era)
- Basic chatbots (Eliza-like, but with more data)
- Machine translation (statistical, phrase-based)

These early NLP methods powered basic applications like keyword search, simple chatbots, and statistical translation tools. While limited by today's standards, they showed the potential of language processing in AI.

Limitations of Classical NLP

- Context windows too small (n-grams)
- Lacks deep semantics or “world model”
- Struggles with nuance, idioms

Classical NLP methods like Bag of Words or n-grams focus on surface-level patterns. They struggle with deeper aspects of language, like capturing context, idioms, or implied meanings. These limitations became clear as applications grew more complex.

Naïve Bayes Classifier

Naïve Bayes (EMC 7.2)

Naïve Bayes

- Probabilistic classification method
- Based on **Bayes' theorem**
- Assumes features are **independent** of each other given the class label
  - It's why we call it **naïve**.
- Method assumes all **features** are **categorical**
  - What if continuous? Can **discretize** based on range
  - Could think of it as "one-hot encoding" for continuous features
  - Note: Extensions exist for continuous features
- Output is the **predicted class label** and a **probability score**

Applications

- Text classification
- Document retrieval
- Spam filtering (0: HAM, 1: SPAM): used at Centre College
- Fraud detection (e.g., insurance industry)

Bayes' Theorem

EMC 7.2.1

Bayes' Theorem (or Rule or even Law!)

- Named for Rev. Thomas Bayes, English mathematician

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}, \qquad P(B) \neq 0$$

- **Conditional probability:** $P(A|B)$
  - Read: "The probability that A is true **given that B is true."**
  - What is the probability student X made A on MAT 310 test?
  - Suppose the student is a Rhodes' Scholar candidate? Or got D in Calc III?

In case someone asks about marginal or joint probabilities:

Suppose we have the posteriors P(x|z) for each case (or class) z.

Compute joint probability P(x, z) = P(x|z)P(z).

Discrete: Sum over cases (classes) z to get the marginal (prior) probability: P(x) = sum_z P(x, z).

If z is continuous (doesn't apply to NB though): Integrate: P(x) = int_z P(x, z)

Both the sum and the integral are over the whole SUPPORT of z.

When we use capital letters, we mean the table of all probabilities for different values discrete variable can take on, e.g., P(A,B) or P(x|Z).

Types of probabilities (don't mix these up!)

- **Prior probability:** $P(A)$ **marginal**
- **Posterior probability:** $P(A|B)$ **conditional** (depends on context)
- **Joint probability:** $P(A,B)$ both A and B true (at same time)

- Convention for discrete random variables (sometimes followed):
  - Lower case variable: P(a|b) specific case
  - Upper case variable: P(A|B) all values of variables (table)
  - Can mix and match: P(A|b), P(b|A), P(a,B), etc.

In case someone asks about marginal or joint probabilities:

Suppose we have the posteriors P(x|z) for each case (or class) z.

Compute joint probability P(x, z) = P(x|z)P(z).

Discrete: Sum over cases (classes) z to get the marginal (prior) probability: P(x) = sum_z P(x, z).

If z is continuous (doesn't apply to NB though): Integrate: P(x) = int_z P(x, z)

Both the sum and the integral are over the whole SUPPORT of z.

When we use capital letters, we mean the table of all probabilities for different values discrete variable can take on, e.g., P(A,B) or P(x|Z).

Rules you should know

- **Sum rule:** $P(A) = \sum_B P(A, B)$
  - Use summation if B is discrete.
  - Use integration (calculus) if B is continuous.
- **Product rule:** $P(A, B) = P(A|B)P(B)$
- **Bayes' theorem:** $P(A|B) = P(B|A)P(A)/P(B)$

General Form for Classification

$$P(C_k|\mathbf{x}) = \frac{P(x_1, x_2, \dots, x_D|C_k)P(C_k)}{P(x_1, x_2, \dots, x_D)}$$

- **Probability of class C_k given** observed feature vector $\mathbf{x}$
- Reminder: $P(x_1, x_2)$ is the **joint probability**.

Independence

- **Important:** $P(A, B) = P(A)P(B)$ only if A and B are **independent**
  - Notation: $A \perp B$
  - **You are not allowed to just go around multiplying probabilities!**
- **Conditional independence:**
  - A and B are said to be conditionally independent given C iff:
  - $P(A|B, C) = P(A|C)$
  - Notation: $A \perp B \mid C$
  - **Can't assume probabilities are independent conditioned on class.**
- **Naïve Bayes classifier assumes you can!!** (hence naïve)

How the Classifier Works

EMC 7.2.2

Naïve Bayes Classifier

- Assumes that probabilities of features
- are conditionally independent
- of all other features
- given the class label.
- Analogy: two bags: think of data as drawn from one or the other
- Given a particular item, which bag is more likely?

$$P(x_1, x_2, \dots, x_D|C_k) = P(x_1|C_k)P(x_2|C_k) \dots P(x_D|C_k) = \prod_{j=1}^{D} P(x_j|C_k)$$

Naïve

- Simplifies computation of $P(x_1, x_2, \dots, x_D|C_k)$
- Also ignores the denominator $P(x_1, x_2, \dots, x_D)$
- Only care about relative probability (which is most likely?)
- Turns out the prediction is directly proportional to true probability
- Also turns out that it is not a very accurate probability!
- But may be good enough (compare perceptron/logistic regression)

Numerical issues

- Probabilities that are exact combinations of features can be very tiny.
- **Numerical underflow** is common.
- Underflow means becomes 0 in CPU, which is very, very, very bad.
- So it is common to work in **log space**.
- Take the log (any base) of all the probabilities
- Then instead of **multiplying**, we can just **add** (slide rule)
- Also note, probabilities don't sum to 1 in this model
- Why does this not matter?

Answer to question: NB classifier may compute $P(y=0|x) = 0.8$ and $P(y=1|x) = 0.7$ and $P(y=2|x) = 0.5$.

Clearly these don't sum to one! But we will just normalize these to 1.0.

The result is we estimate the probability of label 0, 1, and 2 as 0.40, 0.35, and 0.25 respectively.

In other words, we will force things to add up as they should. Again, this is all kind of naïve, but it works in practice for problems such as spam classification or document classification.

Keep in mind that the probabilities you get from this are unlikely to be very reliable. "Ballpark estimates."

Smoothing

EMC 7.2.3

Smoothing

- Recall that in Bayes' thm. probability cannot be 0
- Bad things happen if probability is 0
- If no data that fits an observation, classifier may assume p=0.
- Text model: specific combination of 3 words
- **Laplace smoothing**: also known as **add-one** technique:
- Assumes there is a minimum of at least 1 of everything, or epsilon
- Most packages provide this for you: if not, you must implement it

Assessment

EMC 7.2.4

Assessment

- Discuss: How does it compare with other methods we have seen?
- Easy to implement
- Relatively fast
- Performs well in many practical settings (e.g., spam classification)
- Probabilities not to be taken too literally (but same for logistic regr.)
- As George Box said, "All models are wrong, but some are useful."

NB in Python

PDSH